

WebSquare General Converter

폐쇄망 반입/오프라인 실행용 Source Project 설치·빌드·실행·검증 가이드

프로젝트명: WebSquare General Converter 대상 환경: 인터넷이 없는 폐쇄망 PC (Windows / Linux)

1. 문서 목적

이 문서는 WebSquare General Converter 프로젝트를 폐쇄망 PC로 반입한 뒤, 별도의 인터넷 연결이나 Maven/Gradle 없이 소스를 열고, 빌드하고, sample project를 변환하고, 포함된 offline verifier로 결과를 검증하는 전체 절차를 설명한다.

2. 프로젝트 개요

이 프로젝트는 XPlatform XFDL 화면을 WebSquare XML로 변환하는 Java 8 converter의 **소스 프로젝트**다. 배포용 컴파일 산출물(binary)이 아니라, 폐쇄망에서 직접 컴파일하고 실행할 수 있는 **source project**로 구성되어 있다.

2-1. 표준 변환 경로: TargetWebSquarePipeline (Slice 98BH)

이 절이 현재 표준 조작 경로다. 항목 9(convert-sample.*)와 항목 10의 legacy 8단계 표는 이 프로젝트의 더 이전 Phase4 legacy converter 경로(com.example.xfdltracker.project.XPlatformProjectConverter)를 기술한 **역사적 자료**이며, 현재 accepted standalone 변환/검증 authority가 **아니다**. convert-sample.*는 현재 **non-operational legacy entrypoint로 disabled**되어 있다 -- 실행하면 [CURRENT_PROJECT_CLI_CONFIGURATION_CONTRACT_BLOCKER]와 함께 즉시 종료하며 어떤 변환도 수행하지 않는다(항목 9). 삭제하지 않고 그대로 남겨두었지만, 폐쇄망 standalone 반입 승인의 근거로 사용하지 않는다.

현재 accepted 표준 경로:

```
raw XFDL
→ com.example.xfdltracker.pipeline.TargetWebSquarePipeline.convert(File, File, TargetPipelineConfig)
→ WebSquare XML
```

이 경로는 프로그램적(Java API) 진입점이다 — 호출자가 자신의 실제 대상 환경에 맞는 TargetPipelineConfig/TargetRuntimeProfile을 직접 구성해서 넘겨야 하며, 이 프로젝트는 임의의 "기본" runtime profile을 발명하지 않는다(모든 runtime capability가 사용 가능하다고 가정하는 permissive 기본값도 제공하지 않는다). 여러 파일을 한 번에 처리하는 batch CLI는 Slice 99F에서 closed-network-importWBATCH-CONVERT.cmd/sh + com.example.xfdltracker.batch.ClosedNetworkBatchCli로 제공된다(항목 2-2 참고) — 이 CLI도 임의 대상 환경을 대표하는 단일 "기본" TargetRuntimeProfile을 발명하지 않으며, 호출자가 명시적으로 제공한 profile 파일만 사용한다(파일이 비어 있으면 TargetRuntimeProfile.empty()와 동등하게 처리될 뿐, 암묵적 기본값이 채워지는 것은 아니다).

지원하는 7개 필수 family: GRID, TAB_CONTROL, SPLIT_LAYOUT, SEARCH_AREA, BUSINESS_TABLE, TITLE_BAR, BUTTON_GROUP.

현재 동작 제한(정확히 문서화):

- 일반 스크립트(behavior) 지원 범위는 **유한한 subset**이다 — SourceScriptAnalyzer가 지원하는 top-level 함수 선언 등 닫힌 문법만 지원한다.
- 지원되지 않는 문법은 **fail-closed**한다(변환 전체가 예외로 실패하며, 부분 결과를 발행하지 않는다).
- uc.* 호출은 general behavior lane에서 **번역되지 않는다** — uc.*가 포함된 스크립트는 해당 runtime capability가 실제로 사용 가능해도 general behavior lane에서 항상 fail-closed한다 (GENERAL_SCRIPT_UC_CALL_TRANSLATION_MODEL = FAIL_CLOSED).
- Runtime Option C는 **requirements-only**다 — runtime capability 요구사항을 검증만 하며, 어떤 runtime support 리소스도 발행(emit)하지 않는다.
- 사용 불가능한 runtime capability 요구사항은 **fail-closed**한다(예외 발생, 부분 발행 없음).
- exact target JDK는 여전히 **1.8.0_111**이다. 이 프로젝트의 현재 개발/검증 JDK가 1.8.0_111과 정확히 일치하기 전까지 target-JDK 최종 검증 완료를 주장하지 않는다.

standalone 검증 authority: verify-standalone.bat(Windows). 항목 10의 verify-offline.*는 이제 verify-standalone.bat에 위임하는 thin wrapper로, exact-JDK gate를 포함해 동일한 검증 결과를 낸다(legacy XPlatformProjectConverter/Phase1 SHA verifier 단계는 항목 10 표에 여전히 기술되어 있지만 verify-offline이 더 이상 그 단계들을 실행하지 않는다 — 항목 10 안내를 참고).

2-2. 폐쇄망 batch 변환 endpoint (Slice 99F)

여러 XFDL 파일을 한 번에 변환하려면 closed-network-importWBATCH-CONVERT.cmd(정규 platform, Windows batch, authoritative 구현)를 쓴다:

```
closed-network-importWBATCH-CONVERT.cmd inputRoot outputRoot runtimeProfileFile
```

저장소 root에는 이를 그대로 위임하는 convenience endpoint closed-network-batch-convert.bat (Slice 100D)도 있다:

```
closed-network-batch-convert.bat inputRoot outputRoot runtimeProfileFile
```

이 root wrapper는 caller argument vector와 exit code만 그대로 중계하는 thin delegation wrapper일 뿐이다 -- 새로운 converter나 독립 인증 authority가 아니며, 아래 설명하는 인자 계약/안전 규칙은 모두 BATCH-CONVERT.cmd가 authoritative하게 구현한다(두 경로 모두 최종적으로 동일한 이 구현을 실행한다).

inputRoot 아래에서 확장자가 정확히 .xidl인 일반 파일만 real path 기준으로 재귀 탐색한다(결정적 정렬). **발견된 항목이 심볼릭 링크(Files.isSymbolicLink)로 판명되면 확장자/대상 종류(파일인지 디렉터리인지)/대상이 root 안인지 밖인지와 무관하게 항상 그 자리에서 명시적으로 fail-closed한다**(조용히 건너뛰지 않음, Slice 99F Correction 2). 그와 별개로, 같은 real 디렉터리가 서로 다른 lexical 경로로 두 번 발견되면(root 밖으로 진짜 이탈했든, root 조상으로 되돌아가는 진짜 순환이든, root 안에서 real/과 그리로 향하는 alias처럼 단순히 같은 대상을 두 경로로 노출하는 것이든 전부 포함) 그 자리에서 fail-closed한다 — 어느 경로도 "먼저 왔다고 이기는" 일은 없다. **주의:** Windows junction/reparse alias는 Files.isSymbolicLink()만으로는 감지되지 않는 경우가 있음을 이 machine에서 실측 확인했다(isSymbolicLink()=false, isDirectory()=true로 보고됨) — 그래서 위 real-path 재확인인 symlink 검사와 별도로, 그리고 필수로 존재한다.

inputRoot/outputRoot는 서로 같아서도, 어느 방향으로든 서로 nested되어도 안 되며(모두 real path 기준으로 판정), 상대경로를 보존한 채 outputRoot에 .xml로 확장자만 바꿔 발행하되 계획된 output의 최종 real path가 outputRoot 밖으로 벗어나면 동일하게 거부한다. **그것만으로는 충분하지 않다(Slice 99F Correction 2):** outputRoot부터 목표 파일의 부모 디렉터리까지 이미 존재하는 각 구간의 real path를 순서대로 다시 확인해, 그 중 하나라도 symlink/junction 등으로 실제 다른 곳을 가리키면 -- 그 alias를 통과한 최종 real path가 우연히 여전히 outputRoot 안에 있더라도 -- 발행을 거부한다(아직 없는 구간을 미리 만들어서 검사하지 않음). 각 구간이 "존재하는지"는 NOFOLLOW_LINKS 기준으로 판정한다(Slice 99F Correction 3) -- 대상이 지워진 dangling symlink/junction도 그 자리를 실제로 차지하고 있는 filesystem entry이므로 "아직 안 만들어짐"으로 착각하지 않으며, 심볼릭 링크 구간은 대상 해석과 무관하게 항상 거부하고, junction 등이 real path 해석 자체에 실패하면(끊어진 대상) 그 역시 존재하지 않는 것으로 넘기지 않고 명시적으로 거부한다(이상은 **중간 부모 경로 구간에 대한 검사**). **최종 목표 경로 자체**(정확한 *.xml pathname)도 동일하게 NOFOLLOW_LINKS 기준으로 "이미 점유돼 있는지"를 확인한다(Slice 99F Correction 4) -- 일반 파일/디렉터리/심볼릭 링크/dangling 심볼릭 링크/junction 등 entry 종류와 무관하게 그 이름 자리에 무엇이든 이미 있으면 발행을 거부하며, 대상이 지워진 dangling entry가 그 이름을 차지하고 있어도 "존재하지 않음"으로 오판하지 않는다. **계획된 output 중 하나라도 이미 존재하면(entry 종류 무관) 어떤 개별 pipeline conversion도 시작되기 전에 전부 미리 검사해 명시적으로 fail-closed하며, 기존 entry는 절대 덮어쓰거나 삭제하거나 건드리지 않는다** — 개별 TargetXmlSerializer의 REPLACE_EXISTING 동작은 이 batch 레벨 overwrite 정책의 근거가 아니다. .xidl 입력이 0건이면 공허한 배치를 성공으로 보고하지 않고 명시적으로 실패(종료 코드 2)한다. 이 machine은 실제로 NTFS junction을 만들 수 있어, root 밖 이탈/root 안 별칭/output 중간경로 별칭/dangling 중간경로/**최종 목표 경로 자체를 점유한 dangling entry** 다섯 시나리오 모두 실제 junction으로 회귀 테스트했다(symlink 자체는 이 계정에 권한이 없어 생성 시도 시 건너뛰지만, 코드 경로는 확장자/entry 종류 무관 우선순위로 구성돼 있다).

runtimeProfileFile은 줄마다 정규 capability ID 하나(com.example.xidltracker.runtime.

CommonRuntimeCapabilityCatalog.createSeeded() 기준), #으로 시작하는 줄은 주석, 빈 줄은 무시하는 평문 파일이다(예시: closed-network-importWexample-runtime-profile.txt). 카탈로그에 없는 ID가 하나라도 있으면 변환을 시작하기 전에 명시적으로 fail-closed한다 — 이름으로 capability를 추론하지 않으며, 모든 capability가 사용 가능하다고 가정하는 암묵적 기본 profile도 없다.

이 endpoint는 exact-JDK 게이트/compile/regression 로직을 자체 구현하지 않고 verify-standalone.bat에 그대로 위임한다 — 그 게이트를 통과하지 못하면 batch 변환 자체를 시작하지

않는다(BATCH_CONVERSION_STARTS_BEFORE_TARGET_JDK_GATE = FALSE). 여러 입력 중 하나라도 실패하면 (예: 이미 종결된 CheckBox/GRID-3/Defect 2 계약 한계에 해당하는 입력) 그 시점에서 즉시 멈추고 0이 아닌 종료 코드로 끝나며, 실패한 입력에 대한 부분 산출물은 절대 발행되지 않는다 — 그 이전에 이미 끝난 입력의 output은 그대로 두고 성공/실패/미시도 목록을 그대로 보고한다(부분 완료를 성공으로 위장하지 않음). Legacy 변환기(XPlatformProjectConverter/WebSquareGenerator)는 이 경로 어디에서도 호출되지 않는다. batch 변환이 exit 0로 성공했다는 것이 closed-network-importWMANIFEST.sha256 비교가 현재 HEAD와 일치한다는 뜻은 아니다 — 그

manifest는 별개의 candidate 패키징 스냅샷이며 이 batch 실행 경로의 authority가 아니다(closed-network-import\README-KO.md 항목 7 참고).

저장소 소유 줄바꿈 계약(Slice 99F Correction 5): verify-standalone.bat/closed-network-import\ BATCH-CONVERT.cmd를 비롯한 이 프로젝트의 모든 *.bat/*.cmd는 저장소 루트의 .gitattributes가 -text로 선언한다 — Git이 이 파일들의 줄바꿈을 커밋/체크아웃 어느 방향으로도 절대 변환하지 않고 raw blob 바이트를 그대로 보존한다는 뜻이다. text eol=crlf만으로는 실제 blob 저장 바이트가 바뀌지 않는다는 사실이 외부 실험으로 확인됐으므로(텍스트 파일은 저장 시 항상 LF로 정규화됨) 반드시 -text가 필요하다(Slice 99F Correction 6). -text 선언 자체는 값을 강제하지 않으므로, 이 여덟 개 governed 스크립트(Slice 100D에서 closed-network-batch-convert.bat 추가로 7개에서 8개로 갱신)의 실제 커밋 대상 바이트를 CRLF로 직접 구체화(materialize)해 두었다 — 즉 저장소가 소유하는 계약은 -text 선언과 실제 CRLF 커밋 바이트 두 가지를 모두 포함한다. 이 계약 덕분에 exact-JDK 게이트는 개발자의 core.autocrlf 설정이나 raw Git blob을 어떻게 꺼내 실행하는지와 무관하게 항상 정확히 작동한다(별도의 checkout 정규화나 수동 CRLF 변환을 요구하지 않음).

closed-network-import\BATCH-CONVERT.sh도 있지만 이는 cmd.exe를 거쳐 위 Windows batch를 그대로 호출하는 best-effort 브리지일 뿐이다 — Windows batch(.cmd)가 이 프로젝트의 정규 platform contract이며, .sh는 그 gate를 재구현하지 않고 다만 대신 못 통과시킬 뿐이다(cmd.exe/cygpath를 이 .sh가 찾지 못하면 exact-JDK gate를 통과한 것으로 절대 간주하지 않고 명시적으로 실패한다).

3. 과거 freeze/baseline 기록과의 관계 (참고용 pointer)

이 저장소 자체가 현재 WebSquare General Converter의 독립 source project다. 이 저장소가 파생되어 나온 과거 freeze/baseline 이력(예: Phase4 계열 baseline과 그로부터 파생됐던 이전 candidate 상태)은 이 가이드의 현재 authority가 아니며, 그 historical fact 자체는 OFFLINE-IMPORT-MANIFEST.md(해당 freeze 시점의 historical evidence)와 FROZEN-DO-NOT-MODIFY.md(과거 frozen snapshot의 historical policy record)에 그대로 보존되어 있다.

4. 필수 환경

항목	요구사항
---	---
OS	Windows 또는 Linux/Unix (둘 다 지원 — build.bat/build.sh 등 스크립트 쌍 제공)
JDK	1.8.0_111 (exact)
문자 인코딩	UTF-8
Maven	불필요
Gradle	불필요
외부 JAR	불필요
인터넷 연결	불필요

5. 폴더 구조 설명

```
<project-root>/
├── src/main/java/...      Production Java source 전체 (76개 파일)
├── sample-phase3-project/  135 XFDL + 14 XJS 입력 샘플
├── sample-phase3-output/   보존된 참고용 reference 출력 (136개 XML, accepted runtime/output
│                           authority 아님, 덮어쓰지 않음)
├── audit/                 Python 기반 audit/verifier 스크립트 + Phase1 fixture
├── tools/verifier-src/     Java 기반 Phase1ShaVerifier (Production source와 물리적으로 분리)
├── docs/                  핵심 문서 4종 + 본 가이드(md/docx/pdf)
├── .idea/                  IntelliJ 최소 project metadata (선택)
├── .project / .classpath / .settings/ Eclipse 최소 project metadata (선택)
├── build.bat / build.sh
├── closed-network-batch-convert.bat (root convenience entryptpoint, BATCH-CONVERT.cmd로 위임)
├── convert-sample.bat / convert-sample.sh (non-operational legacy entryptpoint, 비운영 -- 항목 9 참고)
├── verify-offline.bat / verify-offline.sh
├── README-OFFLINE.md
├── OFFLINE-IMPORT-MANIFEST.md
├── SHA256SUMS.txt
└── .gitignore
```

build/(컴파일 산출물)는 스크립트 실행 시에만 생성되는 임시 디렉터리이며, 반입 ZIP 자체에는 포함되지 않는다.

6. IDE 선택 가이드

이 프로젝트는 특정 IDE에 종속되지 않는다. **IntelliJ IDEA / Eclipse / Command Line** 중 자유롭게 선택할 수 있으며, 어느 쪽도 필수 의존성이 아니다. **Command Line**(build.bat/build.sh)이 **최종 기준 빌드 방법**이고, IDE는 이 소스를 열고 실행하는 편의 수단이다.

6.1 IntelliJ IDEA 사용

1. **프로젝트 Open**: IntelliJ에서 File → Open으로 이 저장소의 루트 폴더 자체를 연다(포함된 .idea/ metadata를 인식한다).
2. **Project SDK 설정**: File → Project Structure → Project → SDK에서 폐쇄망 PC에 설치된 **JDK 1.8.0_111**을 선택(없으면 Add SDK로 등록).
3. **Language Level 확인**: 같은 화면에서 Language level이 **8**로 되어 있는지 확인 (.idea/misc.xml에 JDK_1_8로 이미 지정되어 있음).
4. **Source root 확인**: src/main/java가 소스 루트로 인식되는지 확인(.iml에 이미 지정됨).
5. **실행 방법**: IntelliJ 내장 컴파일러로 Build → Build Project를 실행하거나, IntelliJ의 Terminal 탭에서 build.bat/build.sh를 직접 실행해도 된다.

6.2 Eclipse 사용

1. **Import**: File → Import → Existing Projects into Workspace(또는 폴더 구조에 따라 File → Import → Projects from Folder or Archive)로 이 저장소의 루트 폴더를 선택한다(포함된 .project/.classpath를 인식한다).
2. **Installed JRE 등록**: Window → Preferences → Java → Installed JREs에서 폐쇄망 PC의 **JDK 1.8.0_111**을 추가 등록한다.
3. **JavaSE-1.8 Execution Environment 연결**: 같은 Preferences 화면의 Installed JREs 또는 Execution Environments에서 **JavaSE-1.8**에 방금 등록한 JDK 1.8.0_111을 연결한다(.classpath가 JavaSE-1.8 컨테이너를 참조하도록 이미 구성되어 있다).
4. **Compiler compliance 확인**: Window → Preferences → Java → Compiler에서 Compiler compliance level이 **1.8**인지 확인(.settings/org.eclipse.jdt.core.prefs에 이미 지정됨).
5. **source/output folder 확인**: src/main/java가 source, build/classes가 output으로 지정되어 있는지 확인.
6. **실행 방법**: Eclipse 빌드는 저장 시 자동 컴파일되며, Package Explorer에서 원하는 클래스를 Run As → Java Application으로 실행할 수 있다. 또는 Eclipse의 Terminal/외부 셸에서 build.bat/build.sh를 직접 실행해도 된다.

7. IDE 없이 Command Prompt / Shell 사용

방법 C(권장, 가장 단순):

```
JAVA_HOME을 JDK 1.8.0_111 설치 경로로 설정
→ build.bat (Windows) 또는 ./build.sh (Linux/Unix)
→ (변환) TargetWebSquarePipeline.convert(File, File, TargetPipelineConfig) 직접 호출 -- 항목 2-1 참고
→ verify-offline.bat 또는 ./verify-offline.sh
```

주의: convert-sample.bat/convert-sample.sh는 **non-operational legacy entrypoint**(비운영, legacy 진입점)다 — 실행하면 즉시 [CURRENT_PROJECT_CLI_CONFIGURATION_CONTRACT_BLOCKER] 메시지와 함께 종료 코드 1로 종료하며 어떤 변환도 수행하지 않는다(항목 9 참고). 변환은 반드시 TargetWebSquarePipeline을 프로그램적으로(Java API) 직접 호출해야 한다.

8. Build 방법

Windows:

```
build.bat
```

Linux/Unix:

```
./build.sh
```

두 스크립트 모두 다음을 수행한다:

- 현재 JAVA_HOME/java/javac 버전 표시

- 정확히 1.8.0_111이면 [TARGET_JDK_MATCH], 아니면 [TARGET_JDK_MISMATCH_WARNING](빌드 자체는 계속 진행 — 이 경고는 실패가 아니며, target JDK 인증으로 승격되지도 않는다. 인증은 항목 10 참고)
- src/main/java 전체를 build/classes/로 컴파일(소스 트리 자체에는 .class를 생성하지 않음)

9. Sample 변환 방법 -- non-operational legacy entrypoint (비운영, 항목 2-1 참고)

convert-sample.bat/convert-sample.sh는 **현재 비활성화(disabled)된 non-operational legacy entrypoint**다. 이 스크립트들은 과거 legacy XPlatformProjectConverter를 호출했으나, 이제는 실행 즉시 [CURRENT_PROJECT_CLI_CONFIGURATION_CONTRACT_BLOCKER] 메시지를 출력하고 종료 코드 1로 종료한다 — legacy 변환 호출 자체가 스크립트에서 완전히 제거되었다. 어떤 변환도 수행하지 않으므로, **이 절에는 실행 가능한 변환 명령 예시를 더 이상 제공하지 않는다.**

정확한 blocker 사유: 임의의 다중 파일 프로젝트를 한 번에 처리하는 범용 배치 CLI를 위한 Reviewer-approved 기본/일반 TargetRuntimeProfile 정책이 아직 존재하지 않는다(항목 2-1 참고 — 임의 대상 환경을 대표하는 단일 기본 profile을 발명하지 않는다).

현재 accepted 변환 방법은 항목 2-1의 TargetWebSquarePipeline.convert(File, File, TargetPipelineConfig)를 호출자가 자신의 TargetRuntimeProfile과 함께 프로그램적으로 직접 호출하는 것뿐이다.

10. Offline Verification

주의(Slice 98BH hardening 반영): verify-offline.*는 이제 verify-standalone.bat에 위임하는 thin wrapper다 — 아래 8단계 표는 이전(legacy) 버전이 실제로 수행하던 단계를 기술한 역사적 설명이며, 현재 verify-offline.*는 더 이상 convert-sample.* / Phase1 SHA verifier/reference-output diff를 실행하지 않는다. 현재 verify-offline.*가 실제로 수행하는 것은 exact JDK gate(fail-closed) + production/test compile + 전체 project-local test suite 실행이며, 이는 verify-standalone.bat와 동일하다(항목 2-1 참고). exact-JDK 실패 시 시각적으로 여전히 명확히 표시된다.

verify-offline.bat (Windows)
./verify-offline.sh (Linux/Unix)

(legacy) 8단계 구성 -- 역사적 설명:

단계	내용	의미
---	---	---
1	exact JDK 1.8.0_111 게이트	mandatory core gate. java/javac 둘 다 정확히 1.8.0_111이어야 PASS. 다른 8u 버전/JDK11/17/21/--release 8은 인정하지 않음
2	clean compile	build.bat/build.sh 재실행
3	sample conversion	(역사적 설명, 현재 미실행) convert-sample.bat/.sh 재실행, 149/149 확인 -- 이 스크립트는 현재 non-operational legacy entrypoint로 disabled됨(항목 9)
4	생성된 output XML 개수	136개 확인
5	Phase1 SHA verifier	Python(있으면) + Java(항상, mandatory) 양쪽 실행
6	source tree .class/.jar 존재 여부	0/0이어야 PASS
7	reference output diff 요약	build/sample-output/ vs sample-phase3-output/
8	(optional) Node.js JS syntax check	Node 없으면 SKIPPED_OPTIONAL_TOOL

결과 표시는 PASS / FAIL / SKIPPED_OPTIONAL_TOOL 세 가지다. **Python이나 Node가 없어도 SKIPPED_OPTIONAL_TOOL로만 표시되며 core verification 전체를 실패로 만들지 않는다** — 1~4, 5(Java verifier만), 6이 core이고, 8과 5의 Python 부분은 optional이다.

exact JDK 1.8.0_111이 없는 PC에서는 1단계가 반드시 FAIL하고 전체 결과가 [CORE_VERIFICATION_FAIL]로 끝난다 — 이는 정상 동작이며 **converter defect가 아니다**. 다른 JDK로는 target-JDK 인증을 받을 수 없다는 원칙을 이 스크립트가 강제하는 것뿐이다.

11. Phase1 SHA 검증

확정된 추출 recipe(생성된 출력 XML 기준, 소스 XFDL이 아님):

1. 컨버터가 생성한 WebSquare 출력 XML을 대상으로 한다.
2. 문서 내 모든 <script> 요소를 XML 파서(textContent/itertext 의미)로 문서 순서대로 읽는다.
3. 각 요소의 텍스트를 "\n"으로 join한다.
4. 끝의 개행을 제거한 뒤 정확히 1개의 "\n"을 추가한다.
5. UTF-8(BOM 없음)로 인코딩한다.
6. SHA-256을 계산한다.

Expected:

- Sample: f82379cfb619d611ae4137032af43fd10faf3df88f018ca5db3b72c490f4d3fe
- CommentProtection: 14f3466acde50241698ccf21edec5807464a2a6e903854c78ed59332c7b2b987

실행:

```
python audit/phase1_sha_verifier.py audit/phase1_sha_manifest.json
```

또는 (Python 없이, JDK만으로):

```
javac -encoding UTF-8 -d build/verifier-classes tools/verifier-src/com/example/xfdltracker/verifier/Phase1ShaVerifier.java  
java -cp build/verifier-classes com.example.xfdltracker.verifier.Phase1ShaVerifier audit/phase1_sha_manifest.json
```

두 verifier는 동일한 recipe를 구현한다. **주의(역사적)**: 이 절의 Python/Java Phase1 SHA verifier는 legacy 변환 출력에 의존하는 절차이며, 현재 verify-offline.*는 더 이상 이 단계를 자동 실행하지 않는다(항목 2-1/항목 10 참고)
-- 필요하다면 위 명령을 수동으로 직접 실행해야 한다.

12. 현재 검증된 기능

- control structure (id/type/geometry/hierarchy/property): **148 scanned / 146 PASS / 0 MISMATCH**
- geometry / hierarchy: 대표 화면 REAL_RUNTIME_VERIFIED
- Dataset: **13/13**
- Grid: GRID-1(구조+interaction) / GRID-2 REAL_RUNTIME_VERIFIED
- Static / Edit: 정적 value 렌더링 REAL_RUNTIME_VERIFIED
- CheckBox(unbound) — 역사적 widget/bootstrap evidence: **REAL_RUNTIME_VERIFIED** — 과거 legacy WebSquareGenerator 출력 기준으로 addItem(value,label) 호출, 실제 <input> + <label> 생성, click→checked→getValue() round-trip을 실행 엔진에서 확인(역사적 검증 기록, 항목 13-5 참고). **이 검증은 legacy 출력에 대한 것이며 현재 accepted v6 출력의 검증 authority가 아니다.** accepted v6 path는 CheckBox rendering/runtime 동등성이 증명되지 않아(Slice 99E) unbound/dataset-bound 모두 렌더러 도달 전에 명시적으로 fail-closed되며(checkbox_unbound_rendering_equivalence_not_proven), 대상 XML을 전혀 발행하지 않는다(CHECKBOX_UNBOUND_FINAL_DISPOSITION = CLOSED_CONTRACT_LIMITATION, 항목 13-5 참고). accepted path가 ev.onpageload/script bootstrap을 생성하지 않는다는 사실(항목 13-4, Slice 99D)은 이 rendering-equivalence 문제와는 별개이며 그 판정은 그대로 유지된다.
- Phase1 SHA: **STATIC_VERIFIED / PASS**

13. 현재 남은 제한

제품/Runtime known gap(항목 1은 Slice 99A, 항목 2는 Slice 99B correction, 항목 3은 Slice 99C, 항목 4는 Slice 99D, 항목 5는 Slice 99E에서 각각 CLOSED_CONTRACT_LIMITATION으로 종결):

1. Defect 2 — CONTENT_NOT_READY false-negative: **CLOSED_CONTRACT_LIMITATION**(Slice 99A). Tab 동적 navigation(someTab.setUrl(...)/addTab(...) 등)은 identifier.member 형태라 SourceScriptAnalyzer가 항상 UNSUPPORTED_SYNTAX로 결정적으로 거부하며, TargetWebSquarePipeline은 이 시점에 전체 변환을 중단하고 대상 XML을 전혀 발행하지 않는다(TargetWebSquarePipelineTest의 testDefect2TabDynamicSetUrlMemberCallClosedAsContractLimitation/testDefect2TabDynamicAddTabMemberCallClosedAsContractLimitationGeneric, SourceScriptAnalyzerTest의 testDefect2TabDynamicNavigationMemberCallFailsClosedGenerically로 검증). 즉 레거시에서 관찰된 비동기 readiness 오탐(race condition)을 만들어낼 런타임 브리지 자체가 accepted 아키텍처에서는 생성되지 않는다 -- 안전하지 않은 재현 대신 명시적 fail-closed 계약으로 닫힌 상태.
2. GRID-3 — 다중 Format(default/alternate) 정의: **CLOSED_CONTRACT_LIMITATION**(Slice 99B correction). Format이 1개면 그 topology(Columns/Rows/Bands/Cells)가 완전히 파싱/resolve되어 기존과 동일하게

지원된다. Format이 2개 이상 정의돼 있으면 GridFormatParser#resolveFormat은 Format 정의 개수를 감지하고 id 중복만 검사할 뿐, 그중 어느 것이 실제 사용할 "활성 Format"인지 결정적으로 고를 수 있는 source-side selector가 증명되지 않았으므로(id 문자열 의미, 선언 순서, Grid의 어떤 attribute도 evidence 없음 -- Slice 99B correction 재조사 결과) 어떤 Format의 topology도 활성으로 선택/parse하지 않는다 -- "모든 Format이 파싱/검증됐다"는 주장은 하지 않는다. 따라서 Format이 2개 이상이면 활성 topology 선택 이전 단계에서 항상 명시적으로 unresolved로 남고, TargetPayloadExtractor가 렌더러 도달 전에 결정적으로 fail-closed되어 대상 XML을 전혀 발행하지 않는다(GridFormatParserMultiFormatTest, TargetPayloadExtractorTest의 testGridAmbiguousMultiFormatFailsClosedBeforeRenderer, TargetWebSquarePipelineTest의 testIntegrationAmbiguousMultiFormatGridFailsClosedNoPartialOutput로 검증). 이는 "동적 전환 제한"이 아니라 selector 증거 부재에 따른 계약상 한계이며, 향후 source 문법에서 활성 Format selector가 실제로 증명되기 전에는 확장하지 않는다.

3. CheckBox — dataset-bound 케이스: **CLOSED_CONTRACT_LIMITATION**(Slice 99C, correction 2). generic한 `<BindItem compid= propid= datasetid= columnid= />` 선언 문법 자체는 corpus로 증명되어 있다(DatasetBinding.xfdl, Edit 대상). 그러나 **shipped CheckBox 대상 BindItem 실사용례는 corpus/dev pack 어디에도 없으며**, CheckBox의 checked/unchecked 값 semantics와 target w2:checkbox binding/runtime 계약 모두 증명되지 않았다. accepted binding resolution은 evidence-bounded exact-reference subset뿐이다(SourceBindingAnalyzer가 compid를 문서 안 id-attribute exact match로만 resolve, dotted-path 등 미증명 확장 없음). CheckBox를 가리키는 BindItem이 존재하지 않으면(unbound, corpus 유일 사례) 이 dataset-bound 계약-한계 거부 경로 자체는 발동되지 않지만, 그렇다고 파이프라인이 그 CheckBox를 발행하는 것은 아니다 -- 이어서 항목 5(Slice 99E)의 별개 사유(checkbox_unbound_rendering_equivalence_not_proven)로 동일하게 fail-closed된다. source의 어떤 BindItem이 정확히 하나의 CheckBox Element로 exact resolve되면 TargetPayloadExtractor가 그 의미를 추측하지 않고 렌더러 도달 전에 fail-closed된다(checkbox_dataset_binding_no_proven_target_contract). 같은 id를 가진 Element가 2개 이상이라 ambiguous하게 resolve될 때 그 후보 중 하나가 이 CheckBox이면 -- "증명이 없으니 unbound로 넘어간다"가 아니라 -- 별도의 명시적 사유 (checkbox_dataset_binding_component_reference_ambiguous)로 동일하게 fail-closed된다. 무관한 컴포넌트끼리만 얹힌 ambiguous binding은 이 CheckBox에 영향을 주지 않는다(SourceBindingAnalyzerTest, TargetPayloadExtractorTest, TargetWebSquarePipelineTest의 testIntegrationCheckBoxDatasetBoundFailsClosedNoPartialOutput/ testIntegrationCheckBoxAmbiguousBindingFailsClosedNoPartialOutput로 검증). "CheckBox dataset binding source semantics가 완전히 검증됐다"는 주장이 아니라, 정반대로 그 계약이 증명되지 않았다는 사실 자체가 이 종결의 근거다.

4. ev:onpageload 자동 page-init 신뢰성: **CLOSED_CONTRACT_LIMITATION**(Slice 99D). 4가지 별개 질문으로 나누어 판단한다 -- (a) event 선언 생성: accepted v6 path는 ev:onpageload를 **전혀 생성하지 않는다**(AtomicWebSquareRenderer가 target event local name으로 onclick만 인정, TargetPayloadBehaviorFinalizer.EVENT_NAME_MAPPING도 onclick 1건만 존재 -- source onload도 동일하게 UNSUPPORTED_EVENT_MAPPING으로 fail-closed됨); (b) handler body 생성: accepted path에 해당 없음; (c) 실제 자동 발화: 과거(레거시 WebSquareGenerator, pageLoadStatements/addItem 채널) 조사에서도 "확인하지 못함"으로 스스로 기록됐고, 그 채널 자체가 accepted v6 path에서 WebSquareGenerator와 함께 도달 불가; (d) 지원 context 신뢰성: 위 이유로 해당 없음. 즉 accepted path에는 자동 page-init에 의존하는 산출물이 **0건**이다 -- 이 조사 시점(Slice 99D)에는 unbound CheckBox가 빈 `<xf:select appearance="full"/>`로 렌더되어 script/head 콘텐츠 없이 발행되는 pre-99E accepted-v6 attempted structural representation이었고(addItem 호출 없음), BIND-1도 WebSquareGenerator 전용 수정이라 accepted path에 승계되지 않았다. Slice 99E에서 이 unbound CheckBox 경로 자체가 렌더러 도달 전 fail-closed로 단혔으므로(항목 5 참고) 현재는 CheckBox 관련 accepted-path 산출물이 아예 존재하지 않으며, 이 결론(자동 page-init 의존 산출물 0건)은 그대로 유지된다. "자동 발화가 검증됐다"는 주장이 아니라, accepted path가 애초에 이 메커니즘을 만들지 않는다는 구조적 사실이 종결 근거다(TargetPayloadBehaviorFinalizerTest의 testOnloadEventNeverMapsToTargetOnpageload, TargetWebSquarePipelineTest의 testIntegrationCheckBoxUnboundFailsClosedNoPartialOutput/ testIntegrationAllSevenFamiliesReachFinalXml로 생성 XML에 onpageload 부재를 검증).

5. CheckBox(unbound) accepted-v6 rendering-equivalence: **CLOSED_CONTRACT_LIMITATION**(Slice 99E). 역사적 widget/bootstrap evidence(w2:checkbox, addItem(value,label), 실제 `<input> + <label>` 생성, click-checked-getValue() round-trip)는 legacy WebSquareGenerator 출력에 대해서만 확인된 것이다(CHECKBOX_UNBOUND_HISTORICAL_RUNTIME_EVIDENCE = REAL_RUNTIME_VERIFIED_WIDGET_BOOTSTRAP_SEMANTICS). WebSquare dev pack 문서(ROOT/cm/template/snippets/10_입력폼/10_06 Checkbox.xml 등)의 공식 권장 패턴은 `xf:select renderType="checkboxgroup" + xf:choices/xf:item`(다중 항목 그룹 위젯)이며, XPlatform CheckBox(단일 boolean)와 의미가 달라 그대로 재사용할 수 없다 -- Slice 99E에서 감사한 pre-99E accepted-v6 renderer 동작(폐쇄 이전 AtomicWebSquareRenderer가

실제로 만들던 빈 `<xf:select appearance="full"/>`, 자식/ref/item 없음)에 대한 target 문서/실 runtime 근거는 repository 안에 전혀 없었다. real WebSquare runtime 환경이 이 프로젝트에 포함되어 있지 않아(항목 14 참고) 실제 실행 검증도 수행할 수 없었다. 따라서 evidence 없이 "구조적으로 well-formed하니 동작할 것"이라고 추측하지 않고, Slice 99E부터는 unbound CheckBox도 dataset-bound와 동일하게 렌더러 도달 전에 명시적으로 fail-closed한다 -- 그 pre-99E xf:select 구조는 더 이상 accepted path의 현재 산출물이 아니며, 현재 accepted path는 CheckBox(bound/unbound 불문)에 대해 어떤 target XML도 발행하지 않는다 (checkbox_unbound_rendering_equivalence_not_proven, TargetPayloadExtractorTest/AtomicWebSquareRendererTest/TargetWebSquarePipelineTest로 검증). 역사적 검증 기록이 현재 accepted v6 출력의 검증 authority가 되는 것은 아니다 (HISTORICAL_CHECKBOX_RUNTIME_EVIDENCE_IS_ACCEPTED_V6_RENDERING_EQUIVALENCE_AUTHORITY = FALSE). 이는 CheckBox dataset-bound 계약 한계(항목 3)나 ev:onpageload 자동 page-init 종결(항목 4)과는 별개의 fail-closed 사유이며, 이 항목이 항목 4의 auto-page-init 종결을 다시 여는 것은 아니다.

Target JDK 1.8.0_111 인증(Slice 99I):

- exact JDK 1.8.0_111 확보/검증 상태: **PASS**(TARGET_JDK_1_8_0_111_VERIFICATION = PASS, TARGET_JDK_EXACT_RUNTIME_AND_COMPILER_CERTIFIED = TRUE). 로컬에 이미 설치돼 있던 same-home 짝(C:\Program Files\Java\jdk1.8.0_111, java/javac 둘 다 1.8.0_111)을 process-local(비영속) 환경으로 활성화해, 작업 트리/raw committed HEAD의 verify-standalone.bat과 정규/raw committed BATCH-CONVERT.cmd smoke 4건 전부 PASS로 확인했다(상세 근거는 README-OFFLINE.md 항목 7 참고). CLOSED_NETWORK_IMPORT_TECHNICAL_READINESS = READY이나, 이는 위 fail-closed 계약 한계들을 무효화하지 않으며 GitHub Push/공개 배포 완료를 뜻하지도 않는다.

14. WebSquare 배포 관련 주의사항

이 프로젝트에는 다음이 **포함되지 않는다**:

- WebSquare server / WebSquare Studio
- WebSquare engine JAR
- Tomcat
- MariaDB
- license

이 프로젝트는 XFDL → WebSquare XML **변환기** 소스만 제공한다. 생성된 XML을 실제 WebSquare 환경에 배포하려면, 해당 기관 폐쇄망의 WebSquare 환경에 설치된 **wpack 컴파일 도구와 배포 절차**를 별도로 따라야 한다(이 프로젝트가 그 절차를 대신하지 않는다).

15. 오류 해결 / Troubleshooting

증상	원인/조치
---	---
JAVA_HOME 미설정	java/javac가 PATH에 없으면 build.bat/build.sh가 즉시 [FAIL]로 종료. JDK 1.8.0_111의 bin/을 PATH에 추가하거나 JAVA_HOME을 설정
java/javac 버전 불일치	build helper(build.bat/build.sh)는 [TARGET_JDK_MISMATCH_WARNING]만 표시하고 빌드는 계속 진행한다(경고, 실패 아님). 표준 검증 authority(verify-standalone.bat, verify-offline.*는 위임)는 동일 불일치를 [TARGET_JDK_MISMATCH_FAIL]로 fail-closed 처리한다(검증 실패) — exact 1.8.0_111만 인정
JDK가 아닌 JRE만 등록된 경우	javac가 없으므로 컴파일 자체가 불가. JRE가 아닌 JDK 를 설치/등록해야 함
IntelliJ SDK 설정 오류	File → Project Structure → SDKs에서 JDK 1.8.0_111의 실제 설치 경로를 다시 지정
Eclipse Installed JRE 오류	Window → Preferences → Java → Installed JREs에서 등록 상태 확인, JavaSE-1.8 Execution Environment에 연결되어 있는지 재확인
Eclipse compiler level 오류	Window → Preferences → Java → Compiler에서 Compliance level을 1.8로 재설정

증상	원인/조치
UTF-8 문제	모든 스크립트가 -encoding UTF-8/chcp 65001을 명시적으로 사용 — 콘솔 폰트가 한글을 지원하지 않으면 글자가 깨져 보일 뿐 데이터 자체는 정상
build/classes 삭제 후 재빌드	build.bat/build.sh는 매 실행 시 build/를 삭제 후 재생성하므로 별도 수동 삭제 불필요
(역사적, 현재 미해당) Python/Node 미설치 시 optional verification skip	verify-offline.*는 현재 exact JDK gate + compile + 전체 test suite만 수행하며 Python/Node 관련 optional 단계가 없다(항목 10 참고)
generated output과 reference output 차이 확인	verify-offline.*는 이 비교를 더 이상 자동 수행하지 않는다(legacy 변환 출력에 의존하던 단계이며 제거됨, 항목 10) -- 필요하다면 build/sample-output/과 sample-phase3-output/을 직접 diff 도구로 비교

16. 보안/폐쇄망 주의사항

- 인터넷 접근 기능 없음(Production Java 코드가 외부 URL을 호출하지 않고, 스크립트도 curl/wget/download를 실행하지 않음)
- 외부 repository 접근 없음(Maven/Gradle 자체가 없음)
- 실제 credential 포함 안 됨(반입 프로젝트 sanitize 결과는 docs/OFFLINE-DOCUMENT-SANITIZATION.md 참고)
- license 포함 안 됨
- 개발 PC absolute path 의존 없음(모든 스크립트는 SCRIPT_DIR/PROJECT_ROOT 기준 상대경로 사용)

17. 최종 운영 체크리스트

- [] JDK 1.8.0_111 확인 (java -version / javac -version 둘 다 정확히 일치해야 함)
- [] IntelliJ 또는 Eclipse SDK 연결 (또는 Command Line만 사용)
- [] build 성공 (build.bat / build.sh)
- [] verify-standalone.bat(또는 verify-offline.bat/verify-offline.sh) 실행 -- exact JDK 미보유 시 [TARGET_JDK_MISMATCH_FAIL]로 종료하는 것이 정상 동작이다
- [] 실제 변환이 필요하면 TargetWebSquarePipeline.convert(File, File, TargetPipelineConfig)를 자신의 TargetRuntimeProfile과 함께 프로그램적으로 직접 호출한다(항목 2-1) -- convert-sample.bat/convert-sample.sh는 non-operational legacy endpoint이며 실행해도 변환을 수행하지 않는다(항목 9)

아래 항목은 legacy Phase4 baseline 회귀 비교를 위한 **역사적 체크리스트**이며, 현재 convert-sample.*가 disabled되어 더 이상 재현할 수 없다(항목 9 참고, 실행 불가):

- [] (역사적, 재현 불가) sample conversion 149/149
- [] (역사적, 재현 불가) XML 생성 확인 (136개)
- [] (역사적, 재현 불가) Phase1 SHA PASS
- [] (역사적, 재현 불가) reference diff 검토

이 문서(docs/OFFLINE-USER-GUIDE.md)가 canonical source이며, 동일 revision에서 생성된 docs/OFFLINE-USER-GUIDE.docx/docs/OFFLINE-USER-GUIDE.pdf가 함께 제공됩니다.